Intermediate (swap in a structured parser) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `structured_output_practice.py` (Intermediate section): `.with_structured_output(Model)` is LangChain's version of the same constrained-decoding call Doc04 made with `.parse()`. This exercise is where you see that the two are the same idea — including the same trap: a required field forces the model to invent a value. **If not:** the Build Task's `model.with_structured_output(ExtractedInfo)` would be the first time you met LangChain's structured-output call.

Basic Version

Approach 1 — the direct way

```
# structured_output_practice.py — Intermediate section
from pydantic import BaseModel
from langchain_openai import ChatOpenAI


class Person(BaseModel):
    name: str
    age: int


model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
structured_model = model.with_structured_output(Person)

good_result = structured_model.invoke("Extract: Maria is 34 years old.")
print("Clear input:  ", good_result)

unclear_result = structured_model.invoke("Extract: the sky is blue today.")
print("Unclear input:", unclear_result)
```

Expected output (the invented values vary):

```
Clear input:   name='Maria' age=34
Unclear input: name='' age=0
```

No error on the unclear input — the model **had** to fill `name: str` and `age: int`, so it made something up. This is "shape during": the shape is guaranteed, the truth is not.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — `| None` fields, and `extract_person()` returns a `Person` or `None`

```
# structured_output_practice.py — Intermediate section
from pydantic import BaseModel, ValidationError
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI


class Person(BaseModel):
    # why: str | None, not str — Doc04's rule: a required field forces
    # the model to invent a value when the text has none
    name: str | None
    age: int | None


def get_structured_model() -> Runnable:
    # why: .with_structured_output(Person) is LangChain's version of the
    # same constrained-decoding call Doc04 made with .parse()
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return model.with_structured_output(Person)


def extract_person(text: str) -> Person | None:
    structured_model = get_structured_model()
    try:
        person = structured_model.invoke(text)
    except ValidationError:
        # when: rare with "shape during", but a reply can still fail the
        # model — the caller gets None, not a crash
        return None

    # when: nothing found — "no person", not an object full of None
    if person.name is None and person.age is None:
        return None
    return person


def main() -> None:
    texts = [
```

```
        "Extract: Maria is 34 years old.",
        "Extract: the sky is blue today.",
    ]
    for text in texts:
        person = extract_person(text)
        print(f"{text} -> {person}")


if __name__ == "__main__":
    main()
```

Expected output:

```
Extract: Maria is 34 years old. -> name='Maria' age=34
Extract: the sky is blue today. -> None
```

Difference from Basic: `str | None` and `int | None` let the model say "not found" instead of inventing, and `extract_person()` turns that — or a rare `ValidationError` — into `None`, so the caller has exactly two cases.

Which one should you write? Intermediate's — it's the same pattern as Doc04's `extract_info()`, now through LangChain, and it's what the Build Task's chain relies on.